Universidad del Valle de Guatemala
Facultad de Ingeniería
Departamento de Ciencias de la Computación
CC3069 - Computación Paralela y Distribuida

Semestre II - 2026

Proyecto #2 Parte 2 - HPC con OpenMPI colectivo

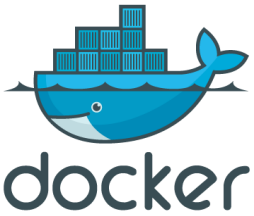
I. Introducción

En la primera fase del proyecto, exploramos cómo MPI (Message Passing Interface) actúa como el estándar fundamental para el modelo de programación paralela en entornos de memoria distribuida. Hasta este punto, la paralelización del algoritmo de fuerza bruta para descifrar un texto en DES se ejecutó simulando procesos paralelos dentro de una misma máquina local (localhost).

Sin embargo, la verdadera fortaleza de MPI radica en su interoperabilidad y portabilidad. El paradigma de intercambio de mensajes nos permite interconectar múltiples máquinas físicas para crear conjuntos computacionales (clusters) altamente escalables, homogeneizando hardware que de otra forma sería incompatible.

En esta Segunda Fase, llevaremos su código a un escenario de infraestructura real. Interconectaremos las computadoras de los distintos miembros de su equipo a través de un enrutador (router) físico. Para superar los desafíos de compatibilidad entre los distintos sistemas operativos de sus computadoras (Windows, macOS, distribuciones Linux), vamos a utilizar contenedores **Docker**. El reto final será desplegar una instancia de Docker, conectarla con las instancias de sus compañeros de grupo y orquestar un ataque de fuerza bruta verdaderamente distribuido para desencriptar el texto provisto por otro equipo de la clase.

II. Objetivos de la segunda parte:



1. **Abstracción de Entorno:** Configurar un entorno virtualizado y estandarizado mediante imágenes de Docker para encapsular las dependencias de compilación y ejecución de OpenMPI (que no dependa de la arquitectura de su computadora).
2. **Orquestación en Red Local:** Lograr la comunicación y el descubrimiento entre contenedores alojados en distintas máquinas físicas a través de una LAN.
3. **Resolución de Problemas (Troubleshooting):** Identificar, diagnosticar y resolver bloqueos de comunicación inter-nodo ocasionados por firewalls, virtualización de redes y enrutamiento.
4. **Análisis Crítico:** Medir y debatir el impacto real que tienen las latencias de red y la arquitectura distribuida sobre los tiempos de *speedup* previamente calculados.



EL OBJETIVO PRINCIPAL DE ESTA SEGUNDA PARTE ES EL TRABAJO EN EQUIPO Y LA DISCUSIÓN DE SUS PROCEDIMIENTOS PARA LOGRARLO.

III. Fase de Validación: Descifrado Cruzado Local

Antes de lidiar con la dificultad de la red distribuida física, es necesario validar la eficacia del algoritmo contra un texto de otro grupo.

1. Ingresen al foro de Canvas y seleccionen el archivo de texto cifrado publicado por otro equipo de la clase, anotando la palabra clave objetivo:

equipo que eligieron	texto	palabra objetivo

2. Ejecuten el programa **bruteforce.c** paralelizado de la Fase 1 localmente (en una sola máquina) para encontrar la llave privada de este nuevo texto.
3. Registren el tiempo de búsqueda base y certifiquen que el texto plano se recupera. Este tiempo de ejecución local será su punto de referencia (T_s) para calcular la eficiencia y los *speedups* al momento de distribuir el trabajo en el clúster.

Log de tiepo y speedup	

IV. Virtualización del Entorno: Preparación con Docker

Para interconectar las computadoras de los integrantes del grupo **eliminando los problemas de compatibilidad entre Windows, macOS y distribuciones de Linux** estandarizaremos el entorno mediante contenedores. Cada integrante construirá una imagen de Docker idéntica que encapsule las dependencias de compilación y los demonios de comunicación.

Deberán redactar un Dockerfile en la raíz de su proyecto con las siguientes especificaciones obligatorias:

1. Dependencias Base: Utilizar una imagen oficial (puede ser ubuntu:22.04) e instalar mediante **apt-get** los paquetes **build-essential, openmpi-bin, libopenmpi-dev y openssh-server.**

2. Comunicación sin Intervención (SSH): MPI requiere conexiones SSH libres (sin necesidad bloqueos) y automatizadas para instanciar

procesos en los nodos trabajadores. En su Dockerfile o script de inicialización, deben alterar las configuraciones del sistema:

- Modificar /etc/ssh/sshd_config para permitir el acceso root (PermitRootLogin yes).

- Deshabilitar la verificación manual de huellas de host en el cliente (StrictHostKeyChecking no en /etc/ssh/ssh_config).

- Ejecución como Root en OpenMPI: Por seguridad, OpenMPI prohíbe la ejecución de procesos iterativos bajo el usuario root. Dado que usaremos contenedores sin volumenes, deben colocar las siguientes variables de entorno para anular esta restricción:
  - ENV OMPI_ALLOW_RUN_AS_ROOT=1
  - ENV OMPI_ALLOW_RUN_AS_ROOT_CONFIRM=1

Validar	**Esperado**	**Funcionó?**	**Que se tuvo que modificar o hacer para lograrlo**
docker run --rm mpi-cluster-image mpicc --version	**deberia mostrar gcc (Ubuntu 11.4.0-1ubuntu1~22.04) 11.4.0**		
docker run --rm mpi-cluster-image mpirun -np 2 echo "Proceso MPI ejecutado"	**deberia imprimir Proceso MPI ejecutado 2 veces**		
crear un contenedor de prueba: docker run -it --rm mpi-cluster-image /bin/bash	**se ejecuta y entra a la terminal del contenedor**		
$ service ssh start	**comienza un**		

	service ssh		
ssh-keygen -t rsa -N "" -f ~/.ssh/id_rsa cat ~/.ssh/id_rsa.pub >> ~/.ssh/authorized_keys	**genera una clave y se prueba de forma local**		
ssh localhost echo "Conexion SSH sin contrasena exitosa"	**imprime el mensaje porque logra levantar una conexion ssh libre**		

V. Despliegue de Red Física y Consideraciones de Topología (Troubleshooting (EL reto real))

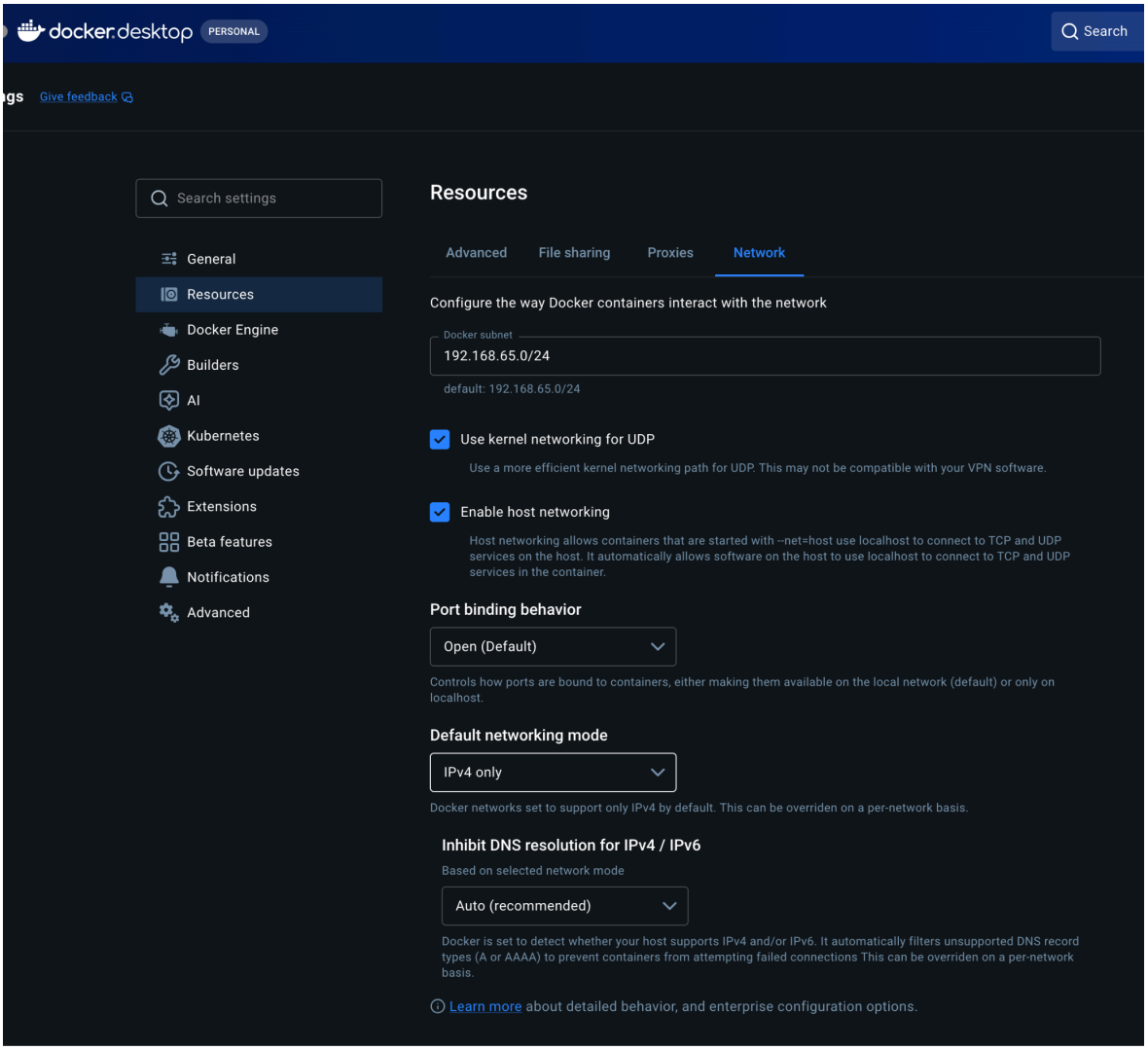
Usando un router, todos los miembros del equipo deben conectarse a esta misma red. Orquestar contenedores a través de múltiples máquinas físicas requiere sortear varias restricciones de red impuestas por los sistemas operativos.

Aislamiento de Red y el Parámetro --network host por default, Docker encierra a los contenedores en una red virtual tipo *Bridge* (**docker0** o **172.17.x.x**), ocultándolos detrás de un NAT. En este estado, los nodos de sus demas compañeros no le van a llegar a su contenedor. Para quitar el aislamiento, **obligatoriamente** deben instanciar su contenedor compartiendo la red de la computadora **host**:

docker run -it --network host --name mpi_node mpi-cluster-image /bin/bash

Limitaciones por Sistema Operativo:

- **Windows (WSL2) y macOS:** Históricamente, la red host no funcionaba fuera de Linux nativo debido a las máquinas virtuales subyacentes. Deben actualizar a **Docker Desktop 4.34 o superior**.



En la configuración global de Docker (Settings > Network), deben habilitar explícitamente la opción **"Enable Host Networking"**.

- **Firewalls (Windows Defender / macOS Firewall):** Al usar la red host, los daemons de OpenMPI intentarán abrir puertos dinámicos entrantes. Sus firewalls de sistema operativo bloquearán silenciosamente este tráfico (provocando errores de *Connection Refused*). Deben configurar reglas para permitir conexiones entrantes en redes de dominio privado.
- **Prevención del "MPI Hang" (Bloqueo Indefinido)** El motor de enrutamiento de OpenMPI inspeccionará todas las interfaces de red de sus computadoras. Si detecta las interfaces virtuales de Docker o de VirtualBox en múltiples máquinas, asumirá erróneamente que puede enviar tráfico criptográfico por ahí. Los paquetes no van a llegar y el programa se va a congelar indefinidamente sin indicar errores. Para evitarlo, al ejecutar el comando en el Nodo Maestro, tienen que forzar a OpenMPI a usar **únicamente** la subred física del router (poe ejemplo **192.168.1.0/24**) mediante parámetros de la Arquitectura de Componentes Modulares (MCA):

  mpirun --hostfile cluster_hosts --mca btl_tcp_if_include 192.168.1.0/24 --mca oob_tcp_if_include 192.168.1.0/24 -np 8 ./bruteforce

- **Gestión del Archivo hostfile y Oversubscription** Al redactar su archivo **hostfile** para indicar las IPs de las computadoras que conecten juntas, es necesario declarar explícitamente el tope físico de recursos mediante **max_slots**. Un algoritmo de fuerza bruta agresivo como DES tiene un *overhead* masivo por cambio de contexto si se le obliga a procesar más hilos que los núcleos lógicos disponibles en el hardware físico

~$ cat mihostfile

```
# Hostfile de ejemplo, los comentarios inician con #
#
# aca un nodo con un solo procesador (slots=1):
foo.example.com
# aca un nodo con dos cores:
bar.example.com slots=2
# aca uno con 4, pero sin permitir
# over-subscribe
yow.example.com slots=4 max-slots=4
# o con las ips de sus companeros:
192.168.1.10 slots=4 max_slots=4

192.168.1.11 slots=8 max_slots=8
```

Una vez que cada miembro del equipo haya superado la validación local y todos estén conectados al router, elijan una computadora como el **Nodo Maestro** y las demás como **Nodos Workers**.

Para validar este punto, el equipo debe documentar y evidenciar el resultado de los siguientes pasos:

Paso A: Sincronización de Llaves SSH inter-nodo Desde el contenedor del **Nodo Maestro**,

generen una llave pública	
cópienla al archivo authorized_keys de los contenedores de los Nodos Trabajadores.	

Paso B: Ejecución del Ataque Distribuido:

Con el archivo **hostfile** correctamente configurado con las IPs locales asignadas por el router (por ejemplo 192.168.1.X), ejecuten desde el Nodo Maestro el comando de paralelización limitando el tráfico a la subred física:

mpirun --hostfile cluster_hosts --mca btl_tcp_if_include 192.168.1.0/24 --mca oob_tcp_if_include 192.168.1.0/24 -np <TOTAL_SLOTS> ./bruteforce

(cambiar la subred segun su IP real)

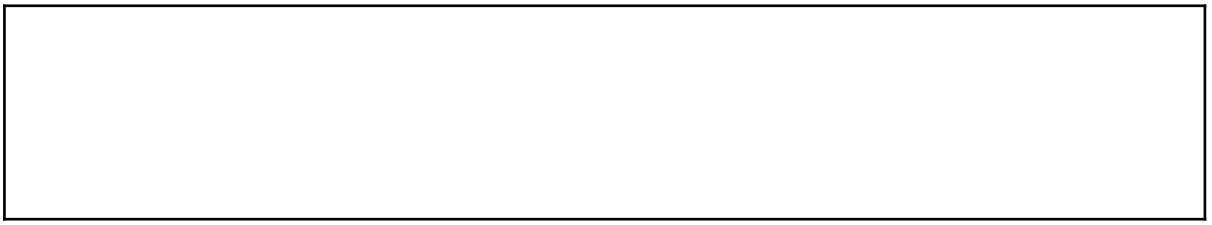
Incluir una captura de pantalla clara donde se observe el comando ejecutado:



la salida en consola mostrando que distintos nodos (IPs) participaron en la búsqueda:



el mensaje final indicando que la llave del otro equipo fue encontrada exitosamente.



VI. Discusión de Resultados y Conclusiones

Adicional a las respuestas que van colocando mientras van realizando las instrucciones, investiguen:

1. Por qué OpenMPI se congela indefinidamente si no se incluyen los parámetros --mca btl_tcp_if_include? Expliquen mecánicamente qué ocurre cuando MPI intenta enrutar tráfico a través de la interfaz puente predeterminada de Docker (docker0) en múltiples máquinas.

2. Detallen los problemas de red que enfrentaron al usar sus computadoras:

3. Comparen el tiempo que les tomó encontrar la llave corriendo el programa localmente vs el tiempo en el clúster físico.

(no tiene que ser en esta tabla necesariamente.)

4. ¿Qué pasa con el rendimiento general de su cluster si configuran el **hostfile** para levantar 32 procesos MPI en una computadora que físicamente solo tiene 4 núcleos?

5. A partir de esta experiencia, resuman las vulnerabilidades y retos logísticos a los que se enfrentan los administradores de sistemas al construir ecosistemas de High Performance Computing (HPC) interconectando hardware heterogéneo para conseguir más recursos de procesamiento:

6. Lograron el objetivo? Cual fue la parte que mas les gusto? Cual fue la que menos les gusto? Que pudieron haber hecho mejor?